

SD Basic - Program Control

Conditions, loops, jumps, and handing control to another program or to the command processor.

This page covers the statements that decide what runs next: conditional execution, loops, jumps, and the several ways one program starts another.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Conditions

```
if expression then statement {else statement}
```

```
if expression then
    statements
end else
    statements
end
```

The single-line and block forms can be mixed — `then` on one line and `end else` opening a block is legal — but a block that starts on one line must end with `end` before the `else`.

An empty branch still needs a statement. Use `null`:

```
read rec from f, id then null else stop 'missing'
```

`and` and `or` have the same precedence and are applied left to right. `a or b and c` means `(a or b) and c`, not `a or (b and c)`. This is not what C, Python, Java or SQL do, and it is the single most common way a condition in ported code changes meaning. **Parenthesise anything that mixes them.** The full precedence table is in SD Basic - Math Functions.

BEGIN CASE

```
begin case
    case expression
```

```
statements
case expression
  statements
case 1
  statements
end case
```

The first matching case runs and the rest are skipped — with two identical `case n = 2` branches, only the first executes. There is no fall-through and no `break`.

`case 1` is the conventional default, because the constant 1 is always true. If no case matches and there is no default, nothing runs.

Loops

COUNTED

```
for variable = start to limit {step increment}
  statements
next variable
```

LOOP	VALUES TAKEN
<code>for i = 1 to 10 step 3</code>	<code>1 4 7 10</code>
<code>for i = 1 to 2 step 0.5</code>	<code>1 1.5 2</code>
<code>for i = 3 to 1</code>	none — the body never runs

The limit is tested before the first pass, so a loop whose start is already past its limit does nothing rather than running once. A fractional `step` is allowed, but a fractional step that cannot represent the limit exactly may stop one iteration early — count with integers and scale if it matters.

`step` may be negative, in which case the loop runs while the variable is greater than or equal to the limit.

CONDITIONAL

```
loop
  statements
while expression
```

```
statements
repeat
```

`while` continues the loop when the expression is true; `until` continues when it is false. Either may appear anywhere in the body, more than once, and a `loop` with neither runs until something in the body leaves it.

`loop / k += 1 / r := k / while k < 3 / repeat` produces `1 2 3` — the test happens where it is written, so the body before it always runs at least once.

```
continue
exit
```

`continue` starts the next pass; `exit` leaves the loop. Both apply to the innermost loop only.

Jumps

```
goto label
go to label
gosub label
return
return to label
```

Labels end with a colon at their definition and are written without one in the jump. A label must be in the same program.

`ON ... GOTO` AND `ON ... GOSUB`

```
on expression goto label1, label2, label3
on expression gosub label1, label2, label3
```

Jumps to the n -th label, counting from 1.

An out-of-range value does not fall through — it is clamped to the nearest end. This is the opposite of what most MultiValue documentation says. On this port, with two labels:

EXPRESSION	WHERE IT WENT
<code>on 0 goto a1, a2</code>	label 1
<code>on 3 goto a1, a2</code>	label 2
<code>on -1 goto a1, a2</code>	label 1

So there is no "none of the above" branch. A value of zero — which is what an empty or non-numeric variable evaluates to — silently runs the first case. **Range-check the expression yourself before the statement**, or use `begin case`, which has a real default.

Stopping

```
stop {message}
abort {message}
quit
```

`stop` ends the program and returns to whatever called it

`abort` ends the program **and everything that called it**, returning to the command processor

`quit` leaves the current command level

`set.exit.status value` sets the value a caller can read, which is how a program run from a script reports success or failure.

Running something else

```
execute expression {capturing variable} {returning variable} {passlist} {rtnlist variable}
perform expression
```

```
chain expression
enter name {(arguments)}
```

`execute` runs a command **at a new level**; the current program resumes afterwards

`perform` as `execute`, but the command shares the caller's level — it can see and change the caller's select lists and files

`chain` replaces the current program; **nothing comes back**

`enter` transfers to another program, also without return, but keeps the current level

`execute 'WHO' capturing cap` puts the command's output in `cap` rather than on the screen, one field per line — `dcount(cap, @fm)` is `1` for a single-session `WHO`.

A command that does not exist is not an error you can catch. `execute 'ZZNOSUCHVERB' capturing out` completes, and `@system.return.code` reads `-1`. The program carries on. **Test** `@system.return.code` **after any** `execute` **whose command name came from data.**

CLAUSE

capturing *var* the command's output goes to *var* instead of the terminal

returning *var* error messages go to *var*

passlist the caller's default select list is handed to the command

rtnlist *var* the command's select list comes back

`chain` **discards unnamed common by default.** The `chain.keep.common` option preserves it — `option chain.keep.common` — which is the setting a ported application usually needs.

Waiting

```
sleep {seconds}
nap milliseconds
pause {timeout}
wake user.number
rqm {time}
```

<code>sleep</code>	wait a number of seconds, or until a time of day
<code>nap</code>	wait a number of milliseconds
<code>pause</code>	wait until another process calls <code>wake</code> , or until the timeout expires
<code>wake</code>	release a process that is paused
<code>rqm</code>	release the processor briefly — the historical "release quantum"

`pause` and `wake` are how two SD sessions coordinate without polling. The user number `wake` needs is the one `@user.no` reports in the waiting process.

Interrupting

```
break {key} {on | off}
break {key} clear
set.break.handler name
remove.break.handler
```

`break off` prevents the user interrupting the program; `break on` restores it. **Every `break off` needs a matching `break on`** — they nest, and a program that aborts between the two leaves the session with interrupts disabled.

`set.break.handler` names a subroutine to run instead of entering the debugger, which is how a program cleans up rather than dying at an arbitrary point.

`break off` around a whole program is a trap, not a safety measure. It also disables the interrupt during any `execute` the program performs, so a runaway query started from inside it cannot be stopped either. Turn it off around the critical section — the write, the transaction — and back on immediately.

Doing nothing

```
null
rem text
remark text
* text
```

`null` is a statement that does nothing, for branches that must be present but empty. `rem`, `remark` and a leading `*` are comments; so is anything after `;*` on a line.

`rem` **is also a function**. `rem(a, b)` is the remainder — see SD Basic - Math Functions. The compiler tells them apart by the bracket: a `rem` immediately followed by `(` is the function, anything else is a comment. `rem (a + b) is wrong` **is a comment**, and `x = rem (a, b)` is a syntax error, because the space changes what it is.

What is not here

These existed in OpenQM or earlier SD releases and **are not in SD Core for Windows**. A program using one will not compile.

<code>aborte</code> · <code>abortm</code>	abort with a message number or text
<code>stope</code> · <code>stopm</code>	stop with a message number or text
<code>rsd</code>	
<code>enter.package</code> · <code>exit.package</code> · <code>package</code>	
<code>trace</code>	
<code>errmsg</code>	see below

`errmsg` **is in the compiler's statement table and still does not compile**. `errmsg 1000` is rejected with "*Unrecognised statement*". The name survives in `BCOMP`'s list but its opcode was removed, so the table is not a reliable guide for this one — **being listed as a statement does not mean a statement exists**.

`procread` and `procwrite` **do** compile, despite appearing on the SD 0.8.0 removed list; `procread` was restored afterwards. Both were checked, not assumed.

See also

SD Basic - Program Structure · SD Basic - Math Functions · SD Basic - File Handling · SD Basic - Select Lists.